

CATAM Additional Projects

7.5 Padé Approximants

1 Introduction

Programming Task

Program A and Program B are listed on pages 18 & 19

2 Estimating functions defined by power series

Question 1

By Taylor expansion about $x = 0$,

$$f_1(x) = f_1(0) + f_1'(0)x + \frac{1}{2}f_1''(0)x^2 + \dots c_k x^k + \dots$$

We have,

$$f_1(x) = (1+x)^{\frac{1}{2}}$$

$$f_1'(x) = \frac{1}{2}(1+x)^{-\frac{1}{2}}$$

$$f_1''(x) = -\frac{1}{4}(1+x)^{-\frac{3}{2}}$$

$\vdots$

and in general, for $k \geq 3$,

$$f_1^{(k)}(x) = (-1)^{k-1} \frac{(2k-3)(2k-5) \dots \times 5 \times 3}{2^k} (1+x)^{-\frac{2k-1}{2}}.$$

Hence,

$$f_1(x) = 1 + \frac{1}{2}x - \frac{1}{8}x^2 + \frac{1}{16}x^3 - \dots + c_k x^k + \dots,$$

where

$$c_k = (-1)^{k-1} \frac{(2k-3)(2k-5) \dots \times 5 \times 3}{2^k k!} \quad (1.1)$$

$$= (-1)^{k-1} \frac{(2k-3)!!}{2^k k!} \quad (1.2)$$

for $k \geq 2$.

To find the radius of convergence we apply the ratio test for power series:

$$\begin{aligned} \left| \frac{c_{k+1}}{c_k} \right| &= \left| \frac{(2k-1)(2k-3) \dots \times 5 \times 3}{2^{k+1}(k+1)!} \frac{2^k k!}{(2k-3)(2k-5) \dots \times 5 \times 3} \right| \\ &= \left| \frac{2k-1}{2(k+1)} \right| \\ &\rightarrow 1 \text{ as } k \rightarrow \infty. \end{aligned}$$

Hence, the radius of convergence is $R = 1$ and the power series converges absolutely for $|x| < 1$. This means that, in theory, the power series will be able to estimate $f_1(x)$ to arbitrary accuracy for $|x| < 1$. The power series diverges for $|x| > 1$ and hence would not be suitable for estimating $f(x)$ in this range.

Noting that the power series does in fact converge for $x = 1$, we consider the partial sums $\sum_{k=0}^N c_k$ as N increases. The evaluated partial sums for N doubling from 5 are displayed in Table 1.1, along with the absolute error $|\epsilon_N| = \left| \sum_{k=0}^N c_k - \sqrt{2} \right|$ and the natural logarithm of this error.

Table 1.1 *The partial sums and their error as an approximation to $\sqrt{2}$ for various N .*

N	$\sum_{k=0}^N c_k$	$ \epsilon_N $	$\log \epsilon_N $
5	1.42578125	0.01156769	-4.45953962
10	1.40993118	0.00428238	-5.45324646
20	1.41266719	0.00154638	-6.47184090
40	1.41366134	$5.52219348 \times 10^{-4}$	-7.50156522
80	1.41401737	$1.96187428 \times 10^{-4}$	-8.53644009
160	1.41414403	$6.95282677 \times 10^{-5}$	-9.57377716
320	1.41418895	$2.46110255 \times 10^{-5}$	-10.61231602
640	1.41420486	$8.70643331 \times 10^{-6}$	-11.65144834
1280	1.41421048	$3.07909290 \times 10^{-6}$	-12.69087552
2560	1.41421247	$1.08878338 \times 10^{-6}$	-13.73044965

To evaluate the coefficients c_k and hence the partial sums, equation (1.1) was used in the form:

$$c_k = (-1)^{k-1} \times \frac{1}{2^2(2)} \times \frac{3}{2(3)} \times \frac{5}{2(4)} \times \dots \times \frac{(2k-5)}{2(k-1)} \times \frac{(2k-3)}{2(k)}$$

which avoided the evaluation of 2^k , $(2k-3)!!$ and $k!$ which quickly grow very large and beyond the upper limit of double-precision floating-point values in MATLAB. In this way it is possible to evaluate the coefficients c_k for arbitrarily large k .

The last column of Table 1.1 suggests a linear relationship between $\log(N)$ and $\log|\epsilon_N|$ which is confirmed by Figure 1.1, where $\log(N)$ and $\log|\epsilon_N|$ follow a near-perfect line.

The equation of the line of best fit is

$$\begin{aligned}\log|\epsilon_N| &= -1.490 \log(N) - 2.024 \\ \Rightarrow |\epsilon_N| &\approx 0.132N^{-1.490}.\end{aligned}\tag{1.3}$$

Indeed, as illustrated in Table 1.2, (1.3) provides a very close approximation for $|\epsilon_N|$.

Figure 1.1 $\log N$ and $\log|\epsilon_N|$ plotted along with the least-squares line of best fit.

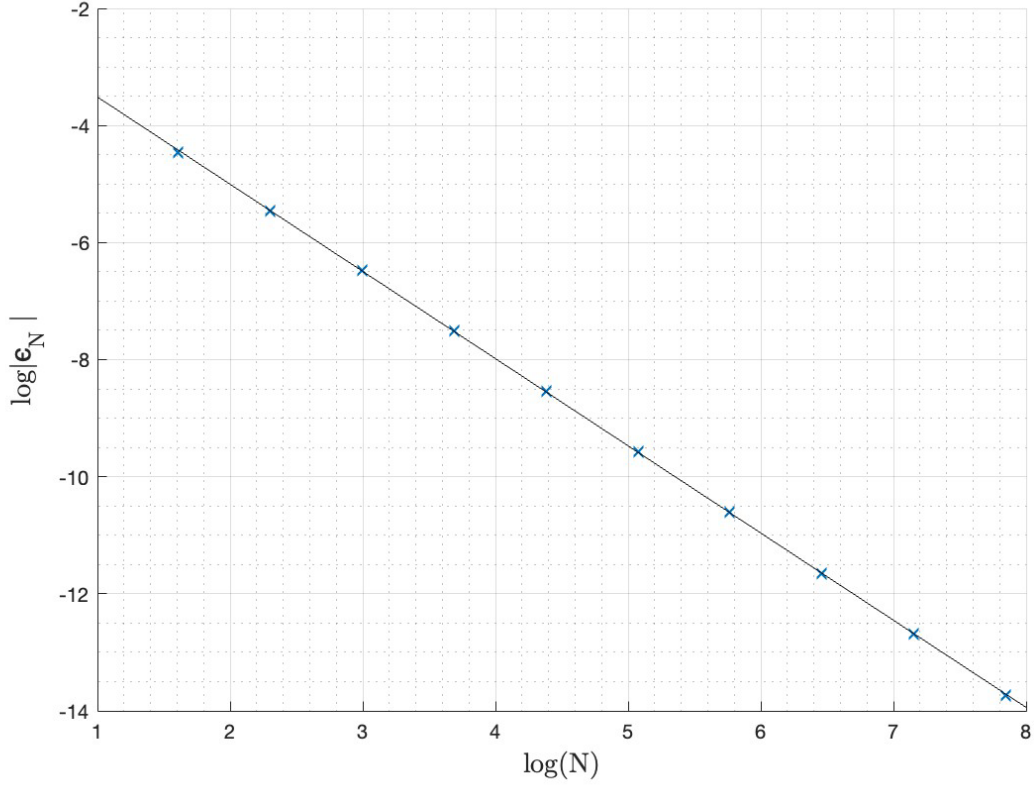


Table 2.2 *The true error and the approximated error.*

N	$ \epsilon_N $	$0.132N^{-1.490}$
5	0.01156769	0.01199799
20	0.00154638	0.00152068
80	$1.96187428 \times 10^{-4}$	$1.92739111 \times 10^{-4}$
320	$2.46110255 \times 10^{-5}$	$2.44287060 \times 10^{-5}$
1280	$3.07909290 \times 10^{-6}$	$3.09621476 \times 10^{-6}$

Question 2

Using Program A to determine $R_{L,L}(x)$ we find:

$$R_{1,1}(x) = \frac{1 + 0.75x}{1 + 0.25x} = \frac{4 + 3x}{4 + x}$$

$$R_{2,2}(x) = \frac{1 + 1.25x + 0.3125x^2}{1 + 0.75x + 0.0625x^2} = \frac{16 + 20x + 5x^2}{16 + 12x + x^2}$$

$$R_{3,3}(x) = \frac{1 + 1.75x + 0.875x^2 + 0.109375x^3}{1 + 1.25x + 0.375x^2 + 0.015625x^3} = \frac{64 + 112x + 56x^2 + 7x^3}{64 + 80x + 24x^2 + x^3}$$

⋮

Evaluations of $R_{L,L}(1)$ for $L = 1$ to 15 are displayed in Table 2.1 along with the error $|\epsilon_L| = |R_{L,L}(1) - \sqrt{2}|$ and the natural logarithm of this error. We can see how $R_{L,L}(1)$ quickly converges towards $\sqrt{2}$ as L increases.

Table 2.1 Initial convergence of $R_{L,L}(1)$ to $\sqrt{2}$ for $L = 1$ to 15.

L	$R_{L,L}(1)$	$ \epsilon_L $	$\log \epsilon_L $
1	1.4	0.01421356	-4.25355867
2	1.413793103448276	$4.20458924 \times 10^{-4}$	-7.77416377
3	1.414201183431953	$1.23789411 \times 10^{-5}$	-11.29951384
4	1.414213197969543	$3.64403552 \times 10^{-7}$	-14.82500395
5	1.414213551646055	$1.07270406 \times 10^{-8}$	-18.35049813
6	1.414213562057320	$3.15774740 \times 10^{-10}$	-21.87599201
7	1.414213562363799	$9.29567534 \times 10^{-12}$	-25.40147184
8	1.414213562372821	$2.73780998 \times 10^{-13}$	-28.92644788
9	1.414213562373087	$8.43769499 \times 10^{-15}$	-32.40606722
10	1.414213562373095	$4.44089210 \times 10^{-16}$	-35.35050620
11	1.414213562373095	$2.22044605 \times 10^{-16}$	
12	1.414213562373095	$< 1.110223 \times 10^{-16}$	
13	1.414213562373095	$4.44089210 \times 10^{-16}$	
14	1.414213562373095	$2.22044605 \times 10^{-16}$	
15	1.414213562373095	$< 1.110223 \times 10^{-16}$	

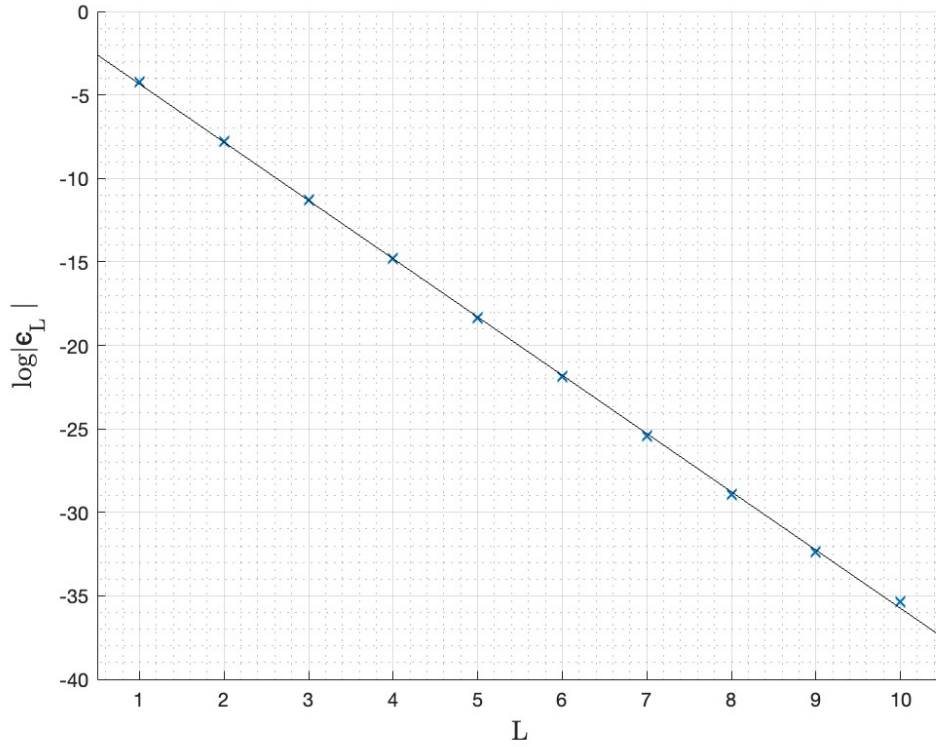
The last column of Table 2.1 suggests a linear relationship between $\log|\epsilon_L|$ and L up to $L = 10$ which is confirmed by Figure 2.1. The equation of line of best fit is:

$$\log|\epsilon_L| = -3.489L - 0.857$$

$$\Rightarrow |\epsilon_L| \approx 0.424e^{-3.489L}$$

Hence, the initial decay of the Padé approximant error is exponential which is much faster than the decay of the power series error.

Figure 2.1 L and $\log|\epsilon_L|$ plotted along with the least-squares line of best fit.



By $L = 10$, the estimate is accurate to 15 decimal places which is the greatest level of accuracy we can expect to achieve. When using double-precision in MATLAB, we can distinguish between numbers (at the scale of $\sqrt{2}$) that differ by at least $\text{eps}_{\sqrt{2}} \approx 2.22 \times 10^{-16}$. Hence, any number we calculate has an uncertainty of $\pm 1.11 \times 10^{-16}$ and so we cannot determine the 16th decimal place accurately.

For $L > 10$ the error $|\epsilon_L|$ fluctuates, remaining around or below 10^{-15} much of the time with a few larger exceptions. The variation in $|\epsilon_L|$ can be attributed to the accumulation of rounding errors whilst solving (4) and (5) which then result in inaccurate values for q_k and p_k . Table 2.2 displays some of the largest values of $|\epsilon_L|$ that arise and the effect of iterative improvement. In each case, iterative improvement reduces the error to around the smallest size we can hope to obtain. The number of iterations was determined manually, terminating when no further reduction in $|\epsilon_L|$ was found. For small values of L ($L < 10$) iterative improvement offers very little, if any, reduction in the errors.

Table 2.2 Effect of iterative improvement on the value of $|\epsilon_L|$ for selected L .

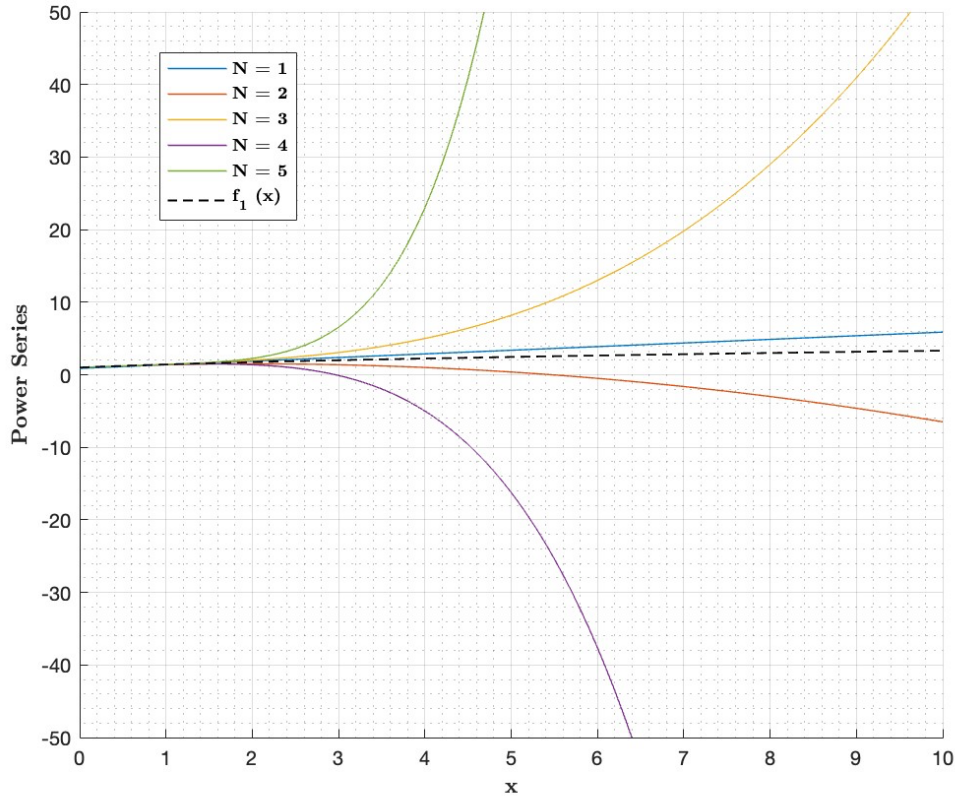
L	$ \epsilon_L $	$ \epsilon_L $ after iterations	Number of iterations
170	$1.96509475 \times 10^{-13}$	$< 1.110223 \times 10^{-16}$	9
421	$6.50590692 \times 10^{-14}$	$< 1.110223 \times 10^{-16}$	3
524	$7.99360577 \times 10^{-14}$	$2.22044605 \times 10^{-16}$	13
559	$2.69118061 \times 10^{-13}$	$4.44089210 \times 10^{-16}$	4

In summary, the Padé approximant gives far more accurate estimates than the power series using far fewer power series coefficients c_k to achieve such estimates. Indeed, using only the first 21 coefficients with $L = 10$, the Padé approximant already gives the most accurate estimate of $\sqrt{2}$ we can expect to achieve. To achieve a similar estimate using the power series would require computation of around 3×10^9 coefficients c_k . Hence it would be recommended to use the Padé approximant to give an estimate for $\sqrt{2}$ to specified accuracy.

Question 3

As discussed in question 1, the power series expansion for $f_1(x)$ does not converge for $x > 1$ and hence cannot be expected to provide valid estimates for $f_1(x)$ for $1 < x \leq 100$. The divergence of the power series is illustrated in Figure 3.1 up to $x = 10$.

Figure 3.1 Power series estimate for $f_1(x)$ for $N = 1, 2, 3, 4$ & 5



In contrast, the Padé approximant $R_{L,L}(x)$ does not diverge for $1 < x \leq 100$ and as shown in Figure 3.2, initially converges to $f_1(x)$ as L increases. As L increases further, $R_{L,L}(x)$ remains close to $f_1(x)$ but does not converge exactly. The closeness of $R_{L,L}(x)$ as an approximation to $f_1(x)$ varies with L and is additionally dependant on the use of iterative improvement. Figure 3.3 plots the error $|\epsilon_L| = |R_{L,L}(x) - f_1(x)|$ against x for various L including two values that gave particularly close approximations. These were $L = 134$ with no iterative improvement and $L = 61$ with 5 iterations.

Figure 3.2 *Padé approximant estimates for $f_1(x)$ for $L = 1, 3, 5, 7, 9$ & 11.*

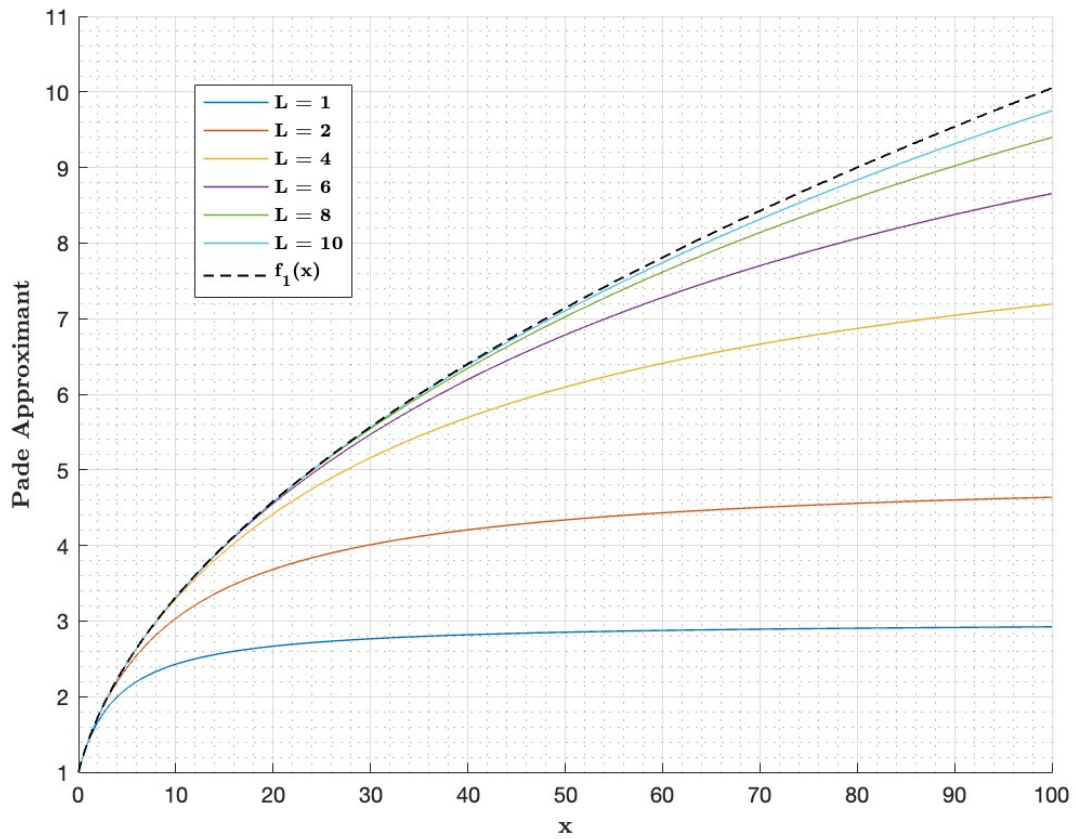
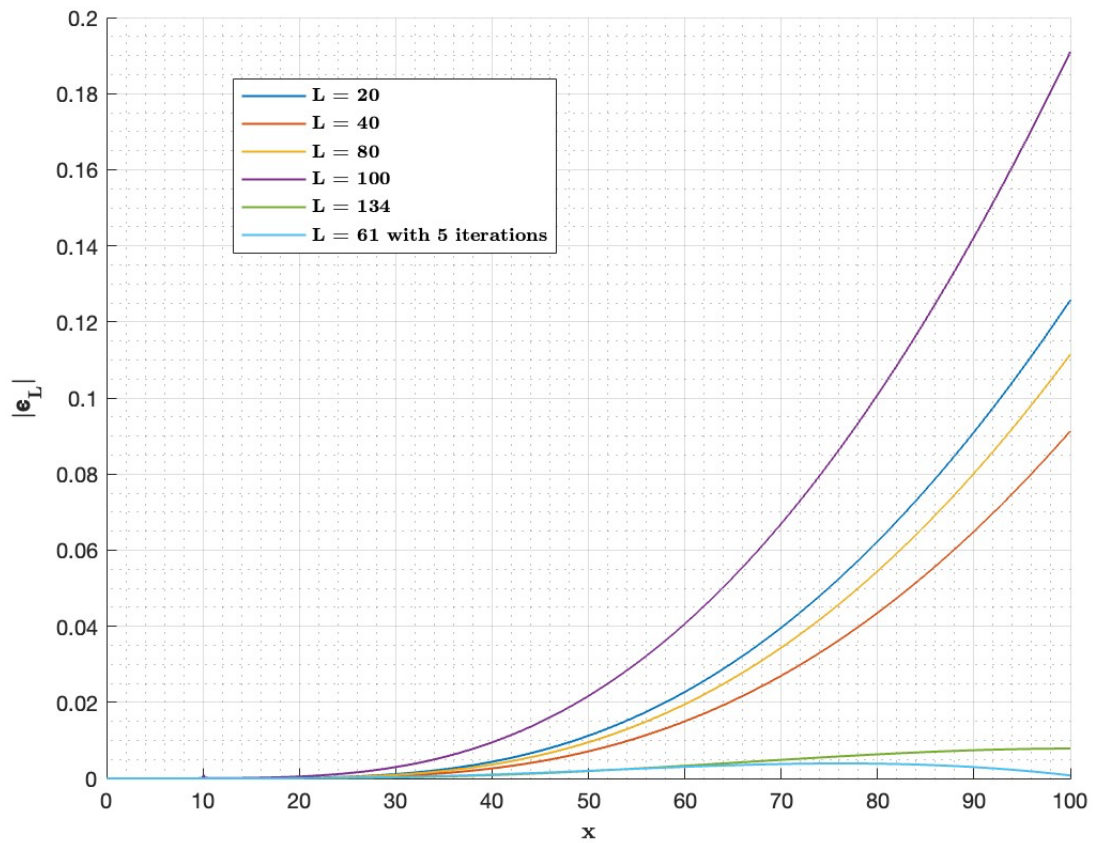


Figure 3.3 *The error $|\epsilon_L|$ of the Padé approximant estimates for various L*



Now looking particularly at $x = 10$ & $x = 100$, we see from Figures 3.4 & 3.5 that in both cases $\log|\epsilon_L|$ decreases linearly with L up to about $L = 12$ beyond which point $\log|\epsilon_L|$ varies about a central value. As with $x = 1$ in question 2, this variation is due to the accumulation of rounding errors whilst solving (4) and (5). Hence, we find once more that the Padé approximant error $|\epsilon_L|$ initially decays exponentially with L . For $x = 10$, the line of best fit is:

$$\log|\epsilon_L| = -1.239L + 1.210$$

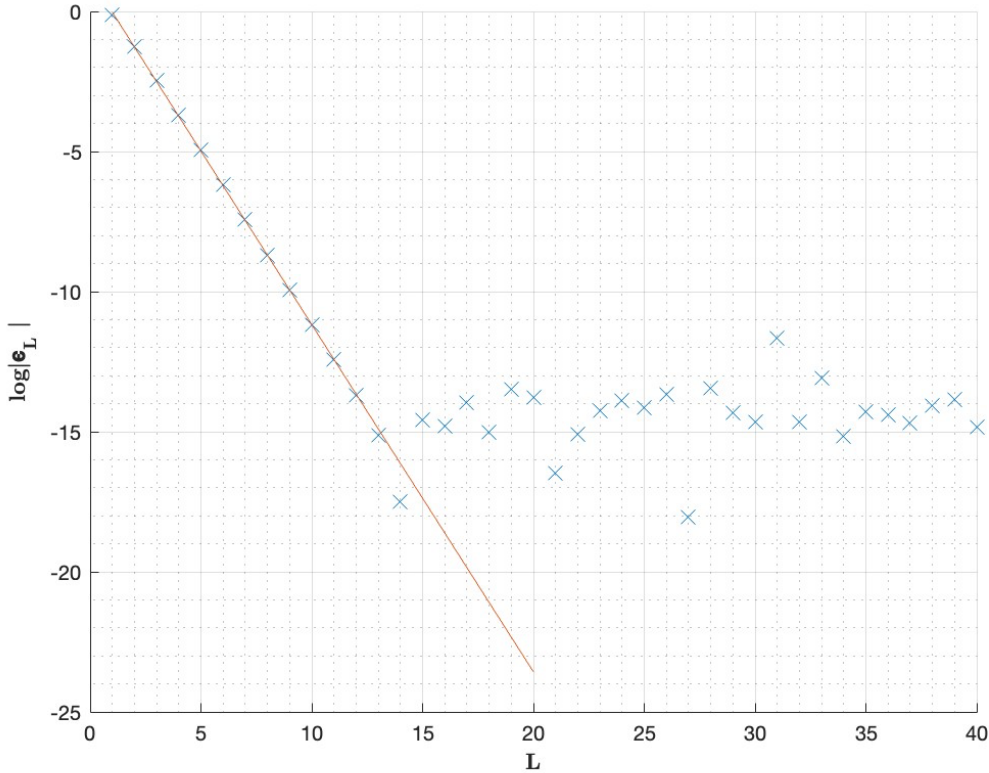
$$\Rightarrow |\epsilon_L| \approx 3.353e^{-1.239L}$$

and for $x = 100$, the line of best fit is:

$$\log|\epsilon_L| = -0.3653L + 2.459$$

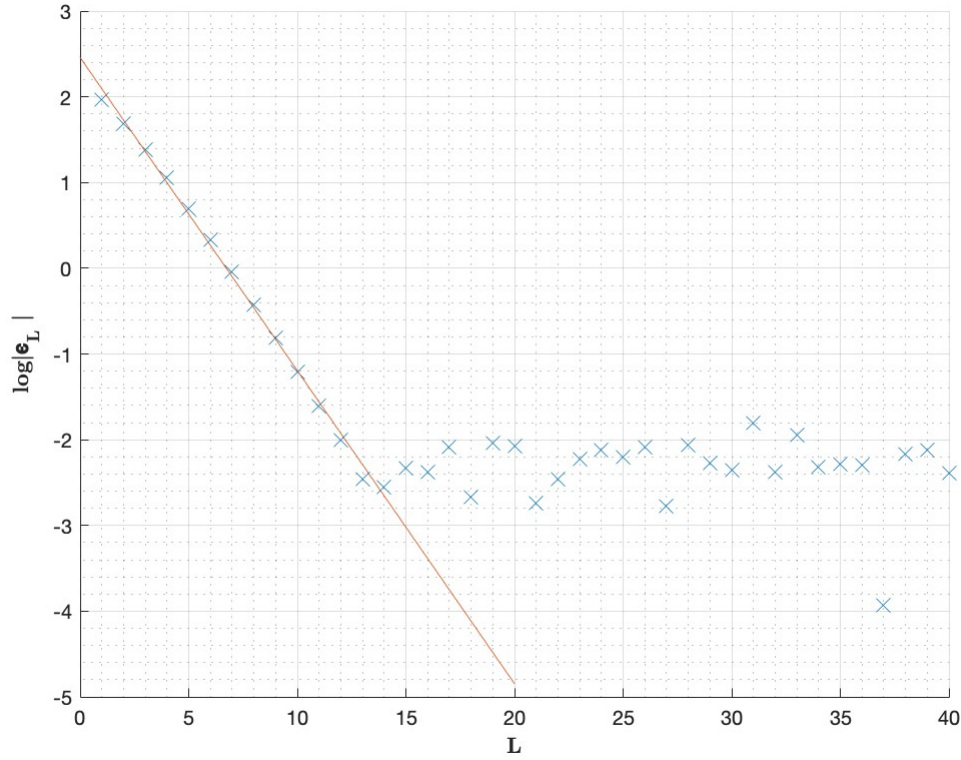
$$\Rightarrow |\epsilon_L| \approx 11.69e^{-0.3653L}$$

Figure 3.4 $\log|\epsilon_L|$ for $R_{L,L}(10)$ as an estimate of $f_1(10)$ for $1 \leq L \leq 40$



In summary, by selecting a particular value of L and using iterative improvement the Padé approximant $R_{L,L}(x)$ can provide good estimates for $f_1(x)$ for larger values of x (e.g. $L = 61$ with 5 iterations). However, for such larger values of x there is a limit to how much ϵ_L can be reduced and the Padé approximant may not be able to estimate $f_1(x)$ to a prior specified accuracy.

Figure 3.5 $\log|\epsilon_L|$ for $R_{L,L}(100)$ as an estimate of $f_1(100)$ for $1 \leq L \leq 40$



Question 4

The truncated power series is only able to give good estimates for $f_2(x)$ when x is very small whilst the Padé approximant provides good estimates for $f_2(x)$ over the full range $0 \leq x \leq 20$. Table 4.1 displays the best truncated power series estimates for $f_2(x)$ for x up to 0.7, beyond which point the best estimate is just the first term, 1. Indeed, from Figure 4.1 we can see that for $x < 0.1$ the truncated power series does provide reasonable estimates for $f_2(x)$.

Table 4.1 N -term truncated power series estimates for $f_2(x)$

x	$f_2(x)$	<i>Truncated power series</i>	N	$ \epsilon_N $
0.01	0.99019422	0.99019422	13	$1.11022302 \times 10^{-16}$
0.03	0.97165496	0.97165496	33	$2.38697950 \times 10^{-14}$
0.05	0.95437091	0.95437090	20	$1.14578003 \times 10^{-8}$
0.07	0.93817425	0.93817131	14	$2.93384148 \times 10^{-6}$
0.09	0.92293158	0.92299313	11	$6.15490782 \times 10^{-5}$
0.1	0.91563334	0.91545632	10	$1.77019398 \times 10^{-4}$
0.2	0.85211088	0.8704	5	0.01828912
0.3	0.80118628	0.88	3	0.07881372
0.4	0.75881459	0.6	2	0.15881459
0.5	0.72265723	0.5	2	0.22265723
0.6	0.69122594	0.4	2	0.29122594
0.7	0.66351027	1	1	0.33648972

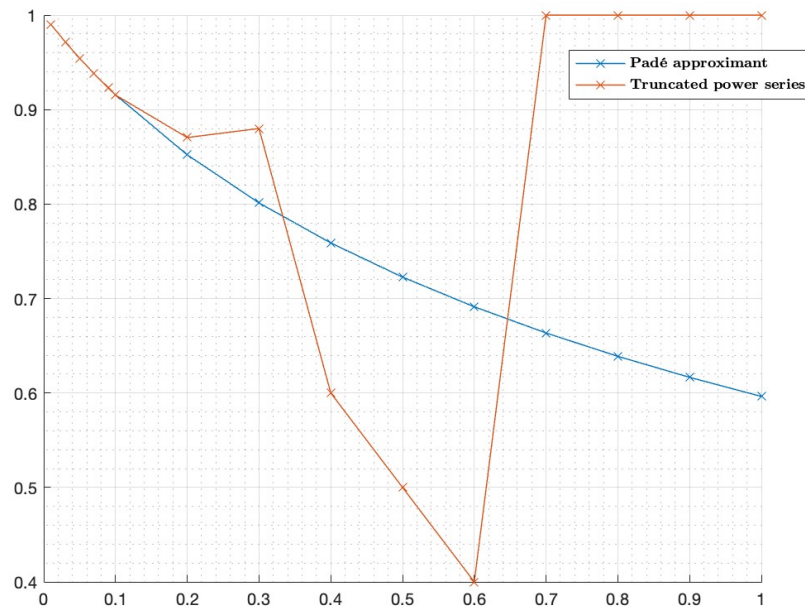
Table 4.2 *Padé approximant estimates for $f_2(x)$.*

x	$f_2(x)$	<i>Padé approximant</i>	$ \epsilon_L $	L
0.1	0.915633339397881	0.915633339397881	$< 5.551115 \times 10^{-17}$	14
0.5	0.722657233776445	0.722657233777786	$1.34092737 \times 10^{-12}$	48
1	0.596347362323194	0.596347362929659	$6.06465433 \times 10^{-10}$	30
3	0.38560201	0.38560243	$4.19935319 \times 10^{-7}$	71
5	0.29866974	0.29867509	$5.34273488 \times 10^{-6}$	71
8	0.22994778	0.22999039	$5.10445628 \times 10^{-5}$	32
10	0.20146425	0.20145908	$5.17043648 \times 10^{-6}$	61
15	0.15652164	0.15663893	$1.17284559 \times 10^{-4}$	70
20	0.12972152	0.13036141	$6.39891935 \times 10^{-4}$	25

Table 4.2 displays Padé approximant estimates for $f_2(x)$ for selected x in the range $0 \leq x \leq 20$ along with the value of L that gave the most accurate estimate after iterative improvement. For each L in the range $1 \leq L \leq 80$ the Padé approximant was calculated using iterative improvement and the most accurate estimate was recorded. Whilst larger values of L generally gave more accurate estimates, the value of L giving the *most* accurate estimate for $f_2(x)$ greatly depended on x . Unlike the power series, the Padé approximant is able to give non-trivial estimates for $f_2(x)$ for larger x , admittedly at a reduced level of accuracy compared to the estimates for smaller x .

Looking at Figure 4.1 it's clear that a comparison between the two methods as a basis for calculating $f_2(x)$ is sensible only for $x < 0.1$. In this range however, the Padé approximant estimates for $f_2(x)$ are exact to working precision in MATLAB (which is 1.110223×10^{-16} at this scale), which is far more accurate than the truncated power series estimates for $x < 0.1$. Hence, considering the accuracy of the two methods, the Padé approximant is the far superior method over the entire range $0 \leq x \leq 20$.

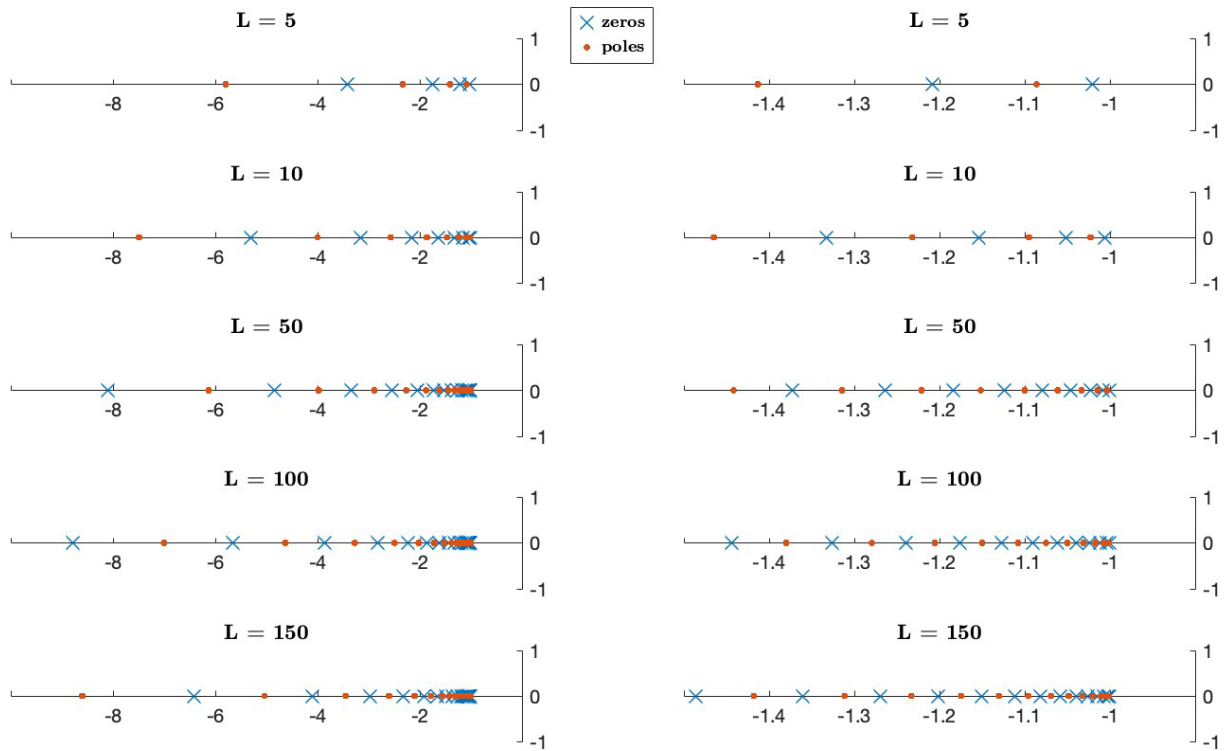
Figure 4.1 *Padé approximant & power series estimates for $f_2(x)$.*



Question 5

As illustrated in Figure 5.1, the poles and zeros of the Padé approximant for $f_1(x)$ all lie along the real axis to the left of -1 , which is a zero and branch point of $f_1(x)$. The poles and zeros alternate along the real axis concentrating near -1 with the point closest to -1 always being a zero. As L increases, this zero gets closer to -1 , which is to be expected as $R_{L,L}(x)$ becomes a better approximation for $f_1(x)$. The points become sparser as we move away from -1 , with the point furthest from -1 always being a pole. As L increases we introduce more poles and zeros, hence increasing their density, especially close to -1 . Apart from the zero closest to $x = -1$, all poles and zeros of $R_{L,L}(x)$ are anomalous in the sense that they do not correspond to poles or zeros of $f_1(x)$.

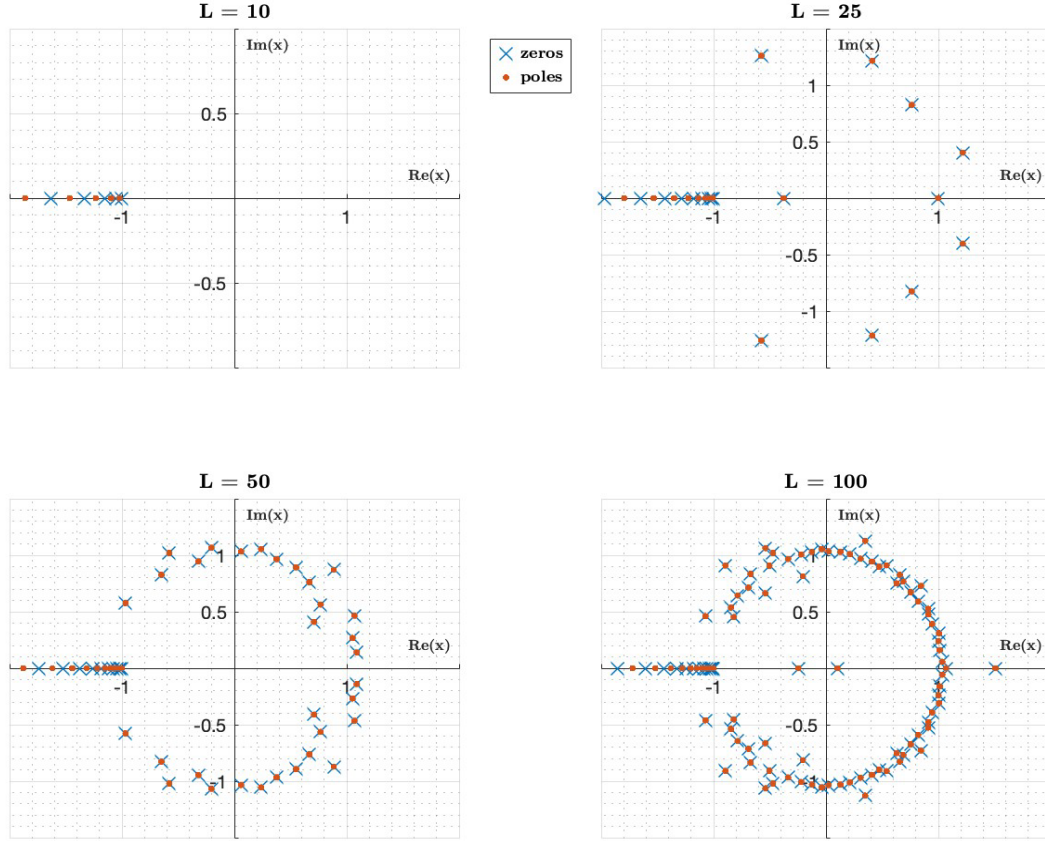
Figure 5.1 Poles and zeros of the Padé approximant for $f_1(x)$.



Omitted in Figure 5.1 are so called ‘defects’ (Baker, G. A., Jr. & Graves-Morris, 1996, p. 48) by which is meant an anomalous pole and zero in very close proximity. These defects are illustrated in Figure 5.2, loosely forming a circle of radius 1 about the origin as L increases. The defects are more transient in nature than the non-defective poles and zeros, appearing and disappearing as L varies.

We see a very similar distribution of poles and zeros for $f_3(x)$ in Figure 5.2 as we did for $f_1(x)$. In fact, just as $f_3(x)$ is the reciprocal of $f_1(x)$, we find that $R_{L,L}(x)$ for $f_3(x)$ is the reciprocal of $R_{L,L}(x)$ for $f_1(x)$. Hence, the poles of $R_{L,L}(x)$ for $f_1(x)$ correspond to the zeros of $R_{L,L}(x)$ for $f_3(x)$ and vice versa. Consequently, Figure 5.3 is the same as Figure 5.1 with the poles and zeros swapped.

Figure 5.2 Poles and zeros (including defects) of the Padé approximant for $f_1(x)$.



For both $f_1(x)$ & $f_3(x)$, all the poles and zeros lie on the line $x \in (-\infty, -1]$ along which we find that the Padé approximant estimate is not valid for any L . This can be understood by recognising that the Padé approximant $R_{L,L}(x)$ is a single valued function and hence can only approximate a particular branch of $f_1(x)$ or $f_3(x)$, which are otherwise both multivalued. For $f_1(x)$ & $f_3(x)$ to be single valued and continuous we must choose a particular branch and hence branch cut which cannot be crossed. Hence, by construction, we expect the Padé approximation to not be valid along this branch cut. In both cases we are approximating the branches of $f_1(x)$ and $f_3(x)$ with branch cut $x \in (-\infty, -1]$.

In the cases where $f(x)$ is multivalued, the branch of f for which our Padé approximant is valid is determined by the power series definition of f which is used to construct the Padé approximant.

Neither $f_4(x)$ or $f_5(x)$ have any branch points, and hence the poles and zeros do not follow any branch cuts, however as we can see in Figures 5.4 & 5.5, the poles and zeros do follow a pattern of sorts. The poles and zeros of $R_{L,L}(x)$ for $f_4(x)$ predominantly follow two curves, with more exceptions as L increases, and there are generally no defects. In contrast, the Padé approximant for $f_5(x)$ has a cluster of defects around origin. In addition, $f_5(x)$ has a pole at $x = -1$ and consequently $R_{L,L}(x)$ has a pole at $x = -1$ for each L . This is unlike the Padé approximant for $f_3(x)$ which has a pole that approaches -1 as L increases but never quite reaches it. We can attribute this to $x = -1$ being a branch point singularity of $f_3(x)$ but only a simple pole of $f_5(x)$.

Figure 5.3 Poles and zeros of the Padé approximant for $f_3(x)$.

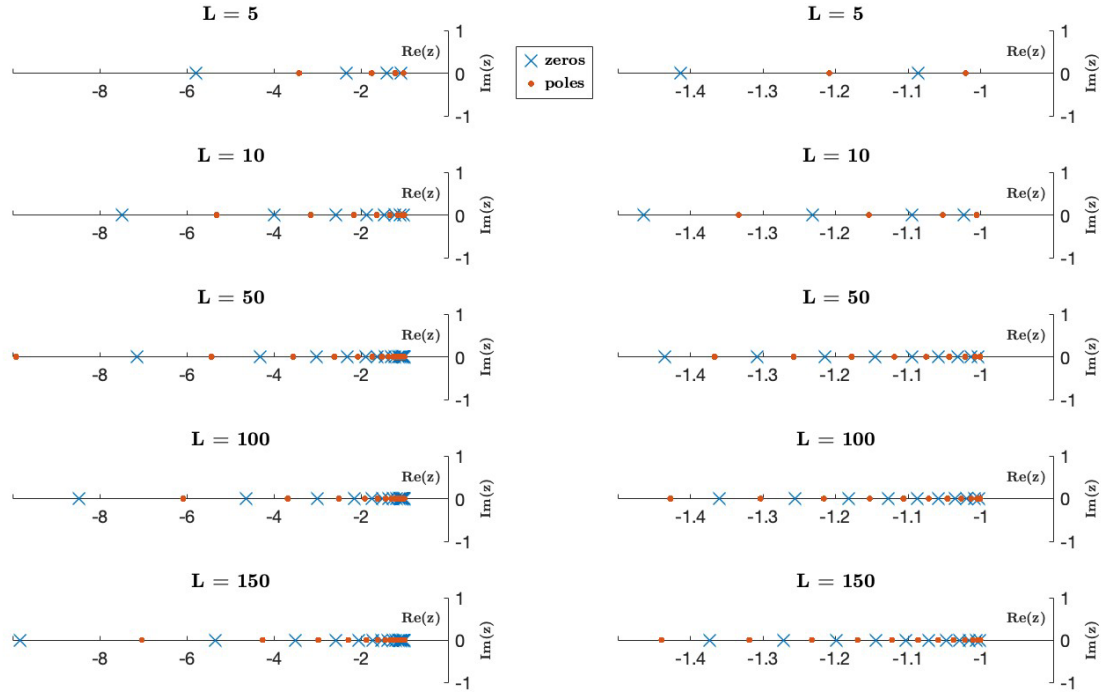


Figure 5.4 Poles and zeros (including defects) of the Padé approximant for $f_4(x)$.

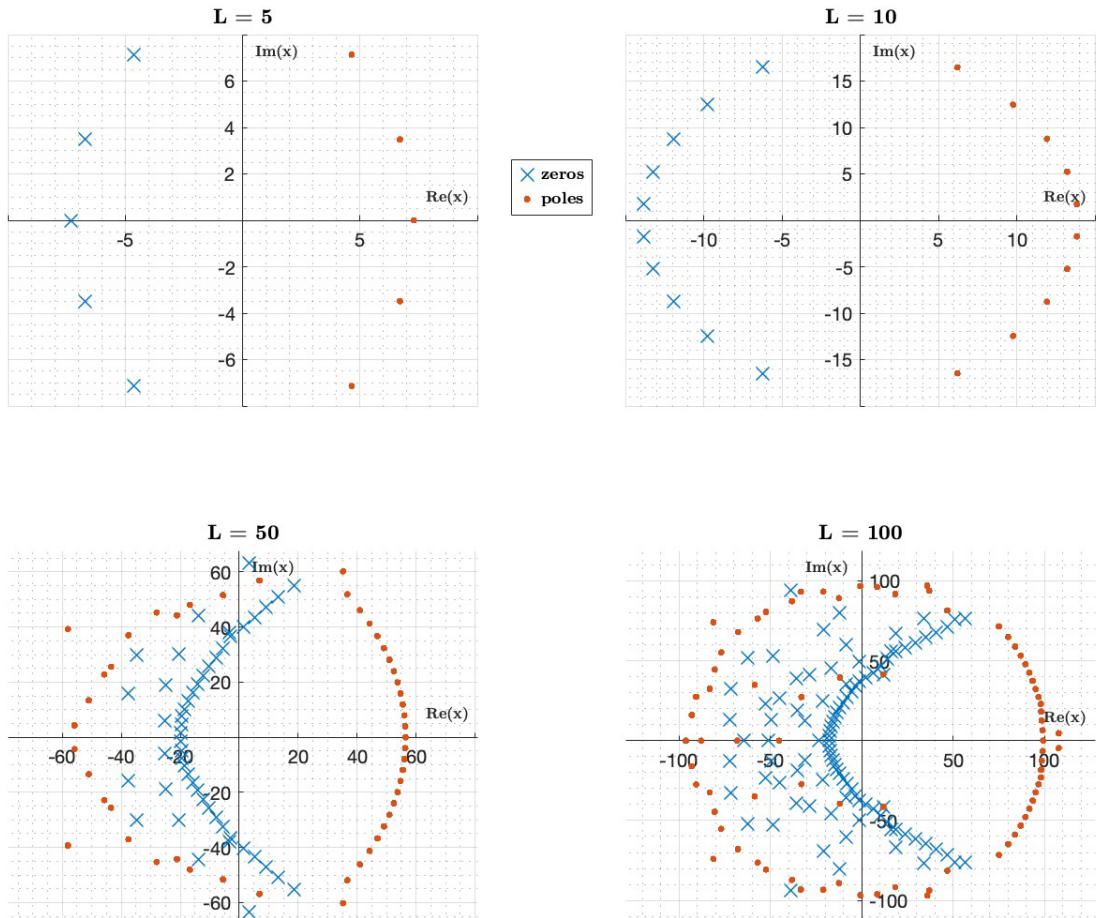


Figure 5.5 Poles and zeros (including defects) of the Padé approximant for $f_5(x)$.

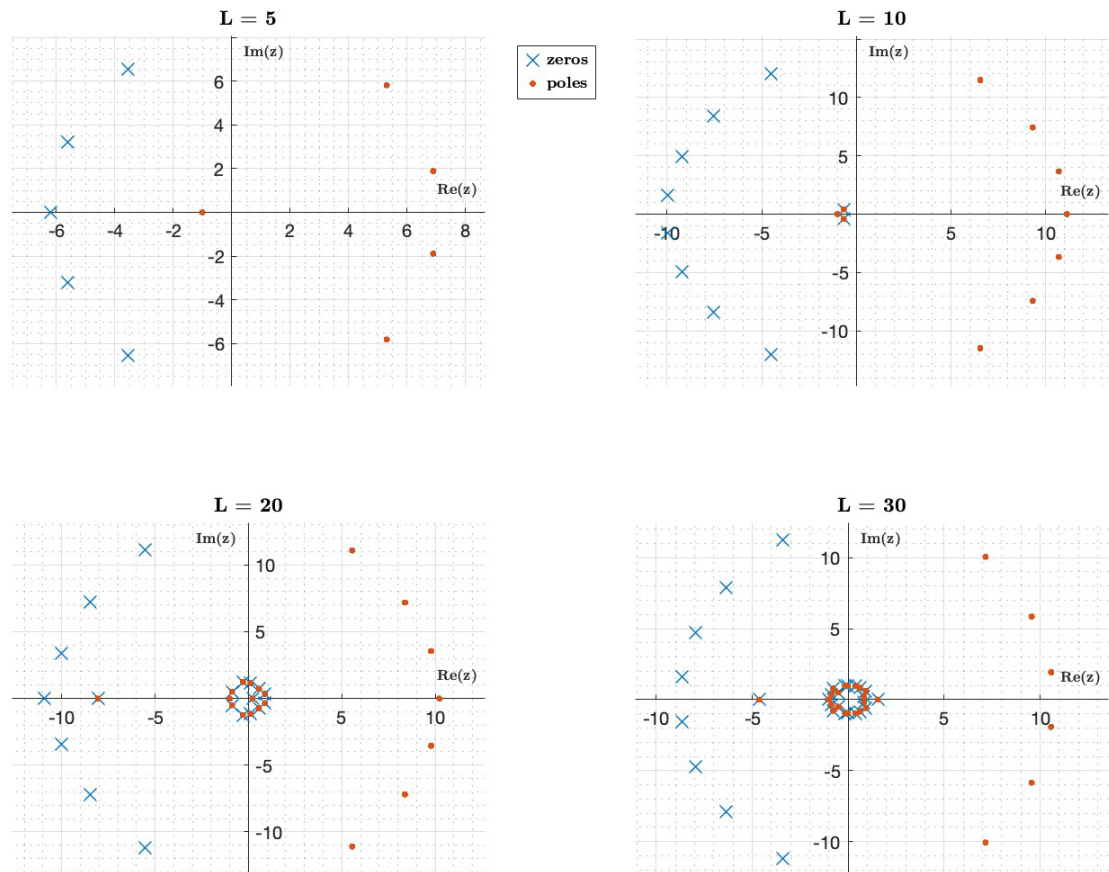
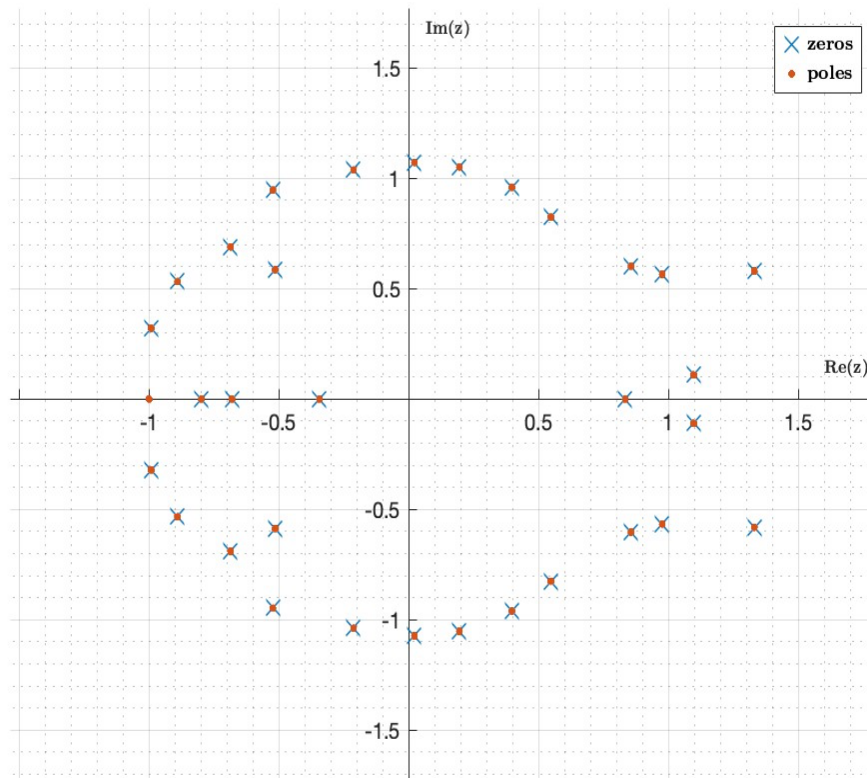


Figure 5.6 Defects near the origin of the Padé approximant for $f_5(x)$ and $L = 40$



As illustrated in Figure 5.7, the poles and zeros of the Padé approximant for $f_6(x)$ follow a well-defined curve in the left half plane, with defects appearing for larger values of L in the right half plane. The non-defective poles and zeros lie along a curve between the two branch points of $f_6(x)$ at $x = -\frac{1}{2} \pm \frac{\sqrt{3}}{2}i$ and increase in density near these branch points. As with $f_1(x)$ and $f_3(x)$, this curve corresponds to the branch cut that defines the branch of $f_6(x)$ for which our Padé approximation is valid. If we stipulate that $f_6(x)$ is single valued, then crossing the branch cut necessarily introduces a discontinuity in $f_6(x)$. As discussed previously for $f_1(x)$ and $f_3(x)$, the Padé approximant itself is a single valued function and hence when evaluating $R_{L,L}(x)$ across the branch cut we inevitably encounter an issue as the Padé approximant accounts for the discontinuity in $f_6(x)$.

We see in Figure 5.8 how the Padé approximant jumps from positive values of $f_6(x)$ to negative values of $f_6(x)$ along the negative real axis. This jump occurs either continuously (in which case $R_{L,L}(x)$ has a zero along the negative real axis) or discontinuously (in which case $R_{L,L}(x)$ has a pole along the negative real axis). Hence, for any L , there will be a significant interval along the negative real axis for which the Padé approximant is not a valid estimate for $f_6(x)$.

Further issues arise from the appearance of defects along the positive real axis for larger values of L , however these only result in inaccurate estimates for $f_6(x)$ in a very small interval around the defect. Since the position of defects changes as L varies we can avoid such issues by selecting a particular value of L that does not have a defect near the point at which we wish to estimate $f_6(x)$.

Figure 5.7 Poles and zeros (including defects) of the Padé approximant for $f_6(x)$.

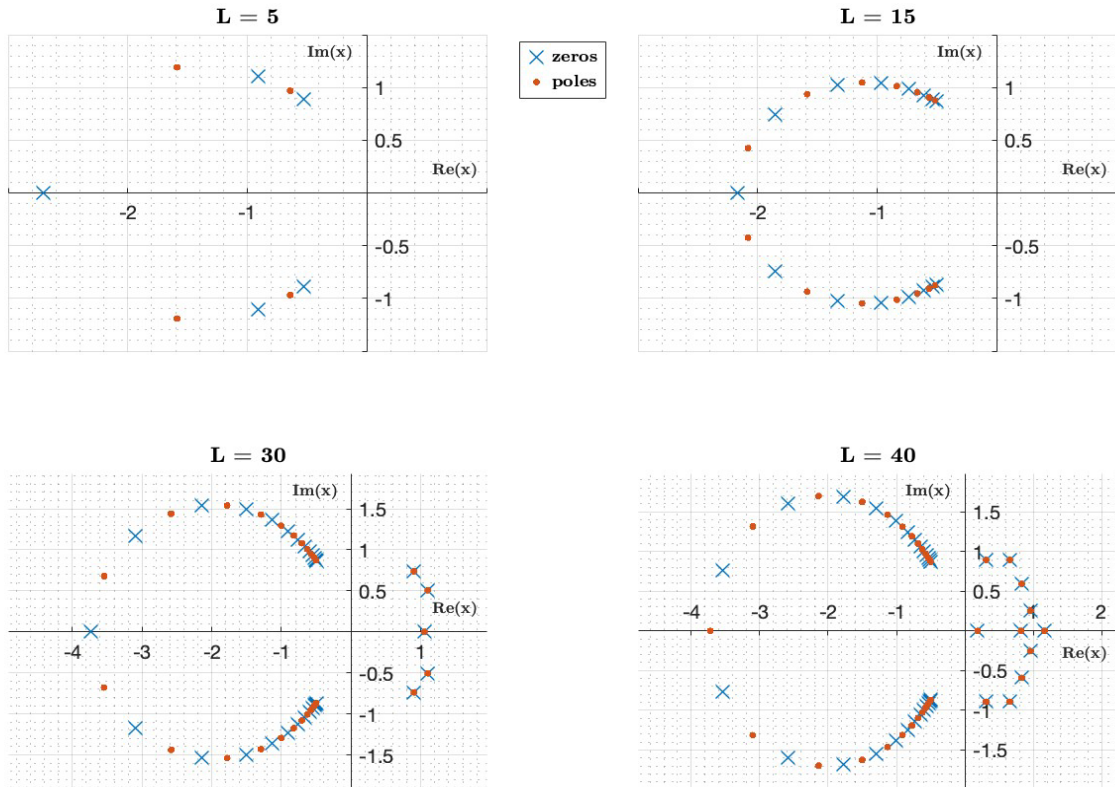
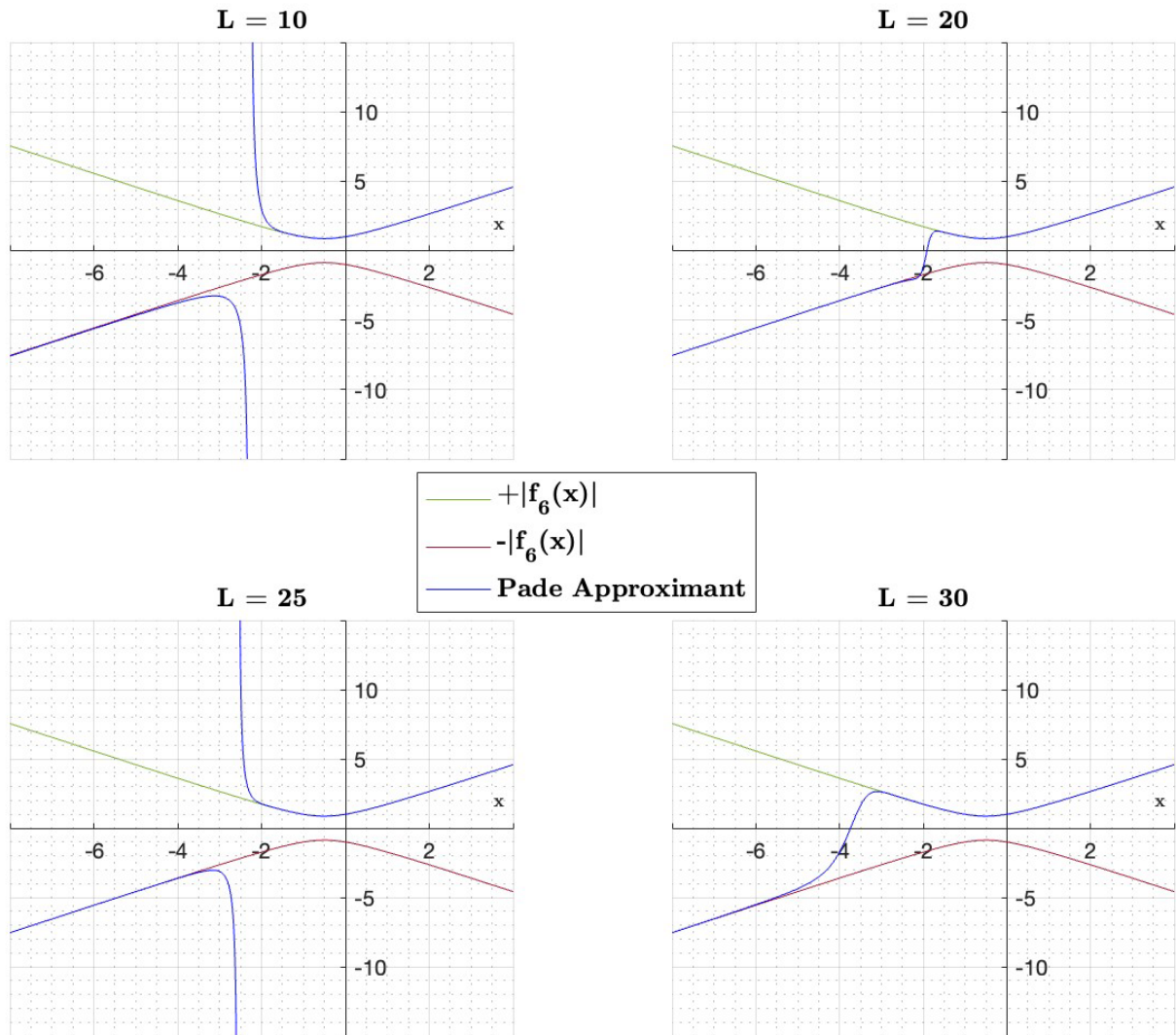


Figure 5.8 Padé approximant for $f_6(x)$ evaluated along the real axis.



End

References

Baker, G. A., Jr. & Graves-Morris, P. *Padé Approximants*. 2nd edition CUP 1996.

Programs

All code written in MATLAB version R2024b with the use of the Symbolic Math Toolbox

Programming Task

Program A:

```
function [p, q, Eval] = ProgramA(c, L, M, N, xvals)
% Solves equations (4) and (5) given the coefficients c_k and the values
% of L and M
% Evaluates the resulting Pade approximant  $R_{(L,M)}(x)$  for a specified set
% of values of x.
% Inputs:
% c - column vector of first L+M+1 coefficients of power series.
% L - value of L.
% M - value of M.
% N - number of iterations for solution
% xvals - x values for the approximant to be evaluated at.
% Outputs:
% q - Column vector of calculated q values.
% p - Column vector of calculated p values.
% Eval - Pade approximant evaluated at points specified by xvals.

cVec = c(L+2:L+M+1);           % establishing equation (4) in matrix form
cMat = zeros(M,M);
for i=1:M
    cMat(:,i) = c(max(L+2-i,1):L+M+1-i);
end
q = mldivide(cMat,-cVec);      % solving (4) for q

for i=1:N                      % iterative improvement
    res = -cVec - cMat*q;
    delta_q = mldivide(cMat,res);
    q = q + delta_q;
end

cMat2 = zeros(L+1,L);         % establishing equation (5) in matrix form
for i=1:L
    cMat2(i+1:L+1,i) = c(1:L-i+1);
end

if M<L
    cMat2(:,M+1:L+1) = [];
end
if M>L
    cMat2(:,L+1:M) = zeros(L+1,M-L);
end
p = c(1:L+1) + cMat2*q;      % solving (5) for p

top = zeros(size(xvals));     % evaluating the numerator and denominator
% of  $R_{(L,M)}$  approximant at xvals
for i=1:L+1
    top = top + p(i)*(xvals.^(i-1));
end
bottom = ones(size(xvals));
for i=1:M
    bottom = bottom + q(i)*(xvals.^(i));
end
Eval = top./bottom;          % evaluation of approximant
end
```

Program B:

```
function [r] = ProgramB(coeff)
% Finds the roots of a polynomial, given the coefficients as input.
% Inputs:
% coeff – The coefficients of the polynomial, in order starting from the
leading order coefficient.
% Outputs:
% r – roots of the polynomial.

r = roots(coeff);

end
```

Question 1

Program used to produce Tables 1.1 & 1.2 and Figure 1.1:

```
N = [5;10;20;40;80;160;320;640;1280;2560];
psum = zeros(10,1);
err = zeros(10,1);

for j=1:10
    c = ones(N(j)+1,1);           % creates vector of coefficients
    c(2) = 0.5;
    c(3) = -0.125;
    for k=3:N(j)                  % calculates each coefficient
        prod = 1/8;
        for i=0:k-3               % implementing equation (1.1)
            prod = prod*(3 + 2*i)/(2*(i+3));
        end
        c(k+1) = ((-1)^(k-1))*(prod);
    end
    psum(j) = sum(c);              % calculates partial sum and error
    err(j) = abs(sqrt(2)-sum(c));
end

scatter(log(N),log(err), 'x',"LineWidth",1.5)
xlabel('log(N)','FontName','CMU Serif')
ylabel('log|e_N|','FontName','CMU Serif')

est = 0.132.*(N.^(-1.490));
table(N, psum, err, log(err), est)
```

Question 2

Program used to produce Table 2.1

```
Eval = ones(15,1);
for L = 1:15
    c = ones(2*L + 1,1);           % creates vector of coefficients
    c(2) = 0.5;
    c(3) = -0.125;
    for k=3:2*L                    % calculates each coefficient
        prod = 1/8;
        for i=0:k-3                % implementing equation (1.1)
            prod = prod*(3 + 2*i)/(2*(i+3));
        end
        c(k+1) = ((-1)^(k-1))*(prod);
    end
    [~, ~, Eval(L)] = ProgramA(c,L,L,0,1);
end
table(transpose(1:15), Eval, abs(Eval - sqrt(2)), log(abs(Eval - sqrt(2))))
```

Program used to produce Table 2.2

```
L = [170;421;524;559];
N = [9;3;13;4];
Eval1 = zeros(4,1);
Eval2 = zeros(4,1);
for j = 1:4
    c = ones(2*L(j) + 1,1);
    c(2) = 0.5;
    c(3) = -0.125;
    for k=3:2*L(j)
        prod = 1/8;
        for i=0:k-3
            prod = prod*(3 + 2*i)/(2*(i+3));
        end
        c(k+1) = ((-1)^(k-1))*(prod);
    end
    [~, ~, Eval1(j)] = ProgramA(c, L(j), L(j), 0, 1);
    [~, ~, Eval2(j)] = ProgramA(c, L(j), L(j), N(j), 1);
end
table(L, abs(sqrt(2)-Eval1), abs(sqrt(2)-Eval2), N)
```

Program used to produce Figure 2.1

```
Lmin = 1;
Lmax = 10;

Eval = zeros(Lmax-Lmin+1,1);

for L = Lmin:Lmax

    c = ones(2*L + 1,1);           % creates vector of coefficients
    c(2) = 0.5;
    c(3) = -0.125;
    for k=3:2*L                   % calculates each coefficient
        prod = 1/8;
        for i=0:k-3               % implementing equation (1.1)
            prod = prod*(3 + 2*i)/(2*(i+3));
        end
        c(k+1) = ((-1)^(k-1))*(prod);
    end
    [p, q, Eval(L-Lmin+1)] = ProgramA(c,L,L,0,1);
end
scatter(Lmin:Lmax, log(abs(Eval - sqrt(2))), 'x', 'LineWidth', 1.5)
xlabel('L', 'FontName', 'CMU Serif')
ylabel('log|εL|', 'FontName', 'CMU Serif')
```

Question 3

Program used to produce Figure 3.1

```
x = 0:0.1:10;
N = 1:5;
hold on
for j=1:numel(N)
    EvalPS = zeros(numel(x),1);
    for t = 1:numel(x)
        c = ones(N(j)+1,1);       % creates vector of coefficients
        c(2) = 0.5;
        c(3) = -0.125;
        for k=3:N(j)              % calculates each coefficient
            prod = 1/8;
            for i=0:k-3            % implementing equation (1.1)
                prod = prod*(3 + 2*i)/(2*(i+3));
            end
            c(k+1) = ((-1)^(k-1))*(prod);
        end
        for k=1:N(j)+1
            c(k) = c(k)*(x(t)^(k-1));
        end
        EvalPS(t) = sum(c);        % calculates partial sum
    end
    plot(x, EvalPS, '-', 'LineWidth', 0.7)
end
plot(x, sqrt(x+1), '--', 'LineWidth', 0.9, 'Color', 'k')
ylabel('Power Series', 'FontName', 'CMU Serif', 'FontWeight', 'bold')
xlabel('x', 'FontName', 'CMU Serif', 'FontWeight', 'bold')
ylim([-50 50])
legend('N = 1', 'N = 2', 'N = 3', 'N = 4', 'N = 5', 'f1(x)',
'FontName', 'CMU Serif', 'FontWeight', 'bold')
```

Program used to produce Figure 3.2

```
x = 0:0.1:100;
L = [1 2:2:10];
hold on
for j = 1:numel(L)
    c = ones(2*L(j) + 1,1);           % creates vector of coefficients
    c(2) = 0.5;
    c(3) = -0.125;
    for k=3:2*L(j)                   % calculates each coefficient
        prod = 1/8;
        for i=0:k-3                 % implementing equation (1.1)
            prod = prod*(3 + 2*i)/(2*(i+3));
        end
        c(k+1) = ((-1)^(k-1))*(prod);
    end
    [~, ~, EvalPA] = ProgramA(c,L(j),L(j),0,x);
    plot(x,EvalPA, '-', 'LineWidth',0.7)
end
plot(x, sqrt(x+1), '--', 'LineWidth', 0.9, 'Color','k')
ylabel('Pade Approximant', 'FontName','CMU Serif', 'FontWeight','bold')
xlabel('x', 'FontName','CMU Serif', 'FontWeight','bold')
legend('L = 1', 'L = 2', 'L = 4', 'L = 6', 'L = 8', 'L = 10', 'f_1(x)',
'FontName','CMU Serif', 'FontWeight','bold')
```

Program used to produce Figure 3.3

```
x = transpose(0:0.1:100);
Lmax = 150;
L = [20 40 80 100 134 61];
c = ones(2*Lmax + 1,1);
c(2) = 0.5;
c(3) = -0.125;
for k=3:2*Lmax
    prod = 1/8;
    for i=0:k-3
        prod = prod*(3 + 2*i)/(2*(i+3));
    end
    c(k+1) = ((-1)^(k-1))*(prod);
end
hold on
for j = 1:numel(L)-1
    [~, ~, EvalPA] = ProgramA(c,L(j),L(j),0,x);
    plot(x,abs(EvalPA-sqrt(x+1)), '-', 'LineWidth',1)
end
[~, ~, EvalPA] = ProgramA(c,L(end),L(end),5,x);
plot(x,abs(EvalPA-sqrt(x+1)), '-', 'LineWidth',1)
ylabel('|εL|', 'FontName','CMU Serif', 'FontWeight','bold')
xlabel('x', 'FontName','CMU Serif', 'FontWeight','bold')
legend('L = 20', 'L = 40', 'L = 80', 'L = 100', 'L = 134', 'L = 61 with 5
iterations', 'FontName','CMU Serif', 'FontWeight','bold')
```


Program used to produce Figure 3.4

```
x = 10;
L = 1:40;

EvalPA = zeros(numel(L), 1);

for j = 1:numel(L)
    c = ones(2*L(j) + 1,1);           % creates vector of coefficients
    c(2) = 0.5;
    c(3) = -0.125;
    for k=3:2*L(j)                   % calculates each coefficient
        prod = 1/8;
        for i=0:k-3                  % implementing equation (1.1)
            prod = prod*(3 + 2*i)/(2*(i+3));
        end
        c(k+1) = ((-1)^(k-1))*(prod);
    end
    [p, q, EvalPA(j)] = ProgramA(c,L(j),L(j),0,x);
end
hold on
plot(L,log(abs(sqrt(1+x) - EvalPA)), 'x','MarkerSize',9)
ylabel('log|ε_L|', 'FontName','CMU Serif', 'FontWeight','bold')
xlabel('L', 'FontName','CMU Serif', 'FontWeight','bold')
X = 1:20;
Y = -1.239*X + 1.21;
plot(X,Y)
```

Program used to produce Figure 3.5

```
x = 100;
L = 1:40;

EvalPA1 = zeros(numel(L), 1);

for j = 1:numel(L)
    c = ones(2*L(j) + 1,1);           % creates vector of coefficients
    c(2) = 0.5;
    c(3) = -0.125;
    for k=3:2*L(j)                   % calculates each coefficient
        prod = 1/8;
        for i=0:k-3                  % implementing equation (1.1)
            prod = prod*(3 + 2*i)/(2*(i+3));
        end
        c(k+1) = ((-1)^(k-1))*(prod);
    end
    [p2, q2, EvalPA1(j)] = ProgramA(c,L(j),L(j),0,x);
end
f1 = figure;
hold on
plot(L,log(abs(sqrt(1+x) - EvalPA1)), 'x','MarkerSize',9)
ylabel('log|ε_L|', 'FontName','CMU Serif', 'FontWeight','bold')
xlabel('L', 'FontName','CMU Serif', 'FontWeight','bold')
X = 0:20;
Y = -0.3653*X + 2.459;
plot(X,Y)
```

Question 4

Program used to produce Table 4.1

```
x = [0.01:0.02:0.09 0.1:0.1:0.7];
N = [13 33 20 14 11 10 5 3 2 2 2 1];

fun = @(t,y) exp(-t).*((1+y.*t).^(-1));

c = zeros(40, 1);
for i = 1:40
    c(i) = ((-1)^(i-1))*factorial(i-1);
end

EvalPS = zeros(numel(N), 1);
for j = 1:numel(x)
    for i = 1:N(j)
        EvalPS(j) = EvalPS(j) + c(i)*(x(j)^(i-1));
    end
end

f2 = zeros(numel(x), 1);

for k = 1:numel(x)
    f2(k) = integral(@(t) fun(t,x(k)),0,Inf);
end

table(transpose(x), f2, EvalPS, transpose(N), abs(f2 - EvalPS))
```

Program used to produce Table 4.2

```
x = [0.1;0.5;1;3;5;8;10;15;20];
Lmax = 80;
c = zeros(2*Lmax + 1, 1);
for i = 1:(2*Lmax + 1)
    c(i) = ((-1)^(i-1))*factorial(i-1);
end
fun = @(t,y) exp(-t).*((1+y.*t).^(-1));
EvalPA = zeros(numel(x), 1);
minL = zeros(numel(x),1);
f2 = zeros(numel(x), 1);
for k = 1:numel(x)
    f2(k) = integral(@(t) fun(t,x(k)),0,Inf);
    PA2 = zeros(Lmax,1);
    Err2 = zeros(Lmax,1);
    for L = 1:Lmax % Iterating over L from 1 to 80
        Err1 = zeros(10,1);
        for i = 1:15 % Iterative improvement up to 15 iterations
            [p1, q1, PA1] = ProgramA(c,L,L,i-1,x(k));
            Err1(i) = abs(f2(k) - PA1);
        end
        [~, ~, PA2(L)] = ProgramA(c,L,L, find(Err1 == min(Err1(:)))-1,x(k));
        Err2(L) = abs(f2(k) - PA2(L));
    end
    [~, Lminim] = min(Err2);
    minL(k) = Lminim;
    EvalPA(k) = PA2(Lminim); % Identifying most accurate estimate
end
table(x, EvalPA ,abs(f2 - EvalPA), minL)
```

Program used to produce Figure 4.1

```
x = [0.01:0.02:0.09 0.1:0.1:1];
N = [13 33 20 14 11 10 5 3 2 2 2 1];
L = 80;
c = zeros(2*L + 1, 1);
for i = 1:(2*L + 1)
    c(i) = ((-1)^(i-1))*factorial(i-1);
end
fun = @(t,y) exp(-t).*((1+y.*t).^(-1));
EvalPA = zeros(numel(x), 1);
EvalPS = [zeros(numel(N), 1); ones(numel(x)-numel(N), 1)];
f2 = zeros(numel(x), 1);
for k = 1:numel(N)
    f2(k) = integral(@(t) fun(t,x(k)),0,Inf);
    Err = zeros(20,1);
    for i = 1:20
        [p1, q1, PA] = ProgramA(c,L,L,i-1,x(k));
        Err(i) = abs(f2(k) - PA);
    end
    [p2, q2, EvalPA(k)] = ProgramA(c,L,L, find(Err == min(Err(:)))-
1,x(k));
    for i = 1:N(k)
        EvalPS(k) = EvalPS(k) + c(i)*(x(k)^(i-1));
    end
end
for k = numel(N)+1:numel(x)
    f2(k) = integral(@(t) fun(t,x(k)),0,Inf);
    Err = zeros(20,1);
    for i = 1:20
        [p1, q1, PA] = ProgramA(c,L,L,i-1,x(k));
        Err(i) = abs(f2(k) - PA);
    end
    [p2, q2, EvalPA(k)] = ProgramA(c,L,L, find(Err == min(Err(:)))-
1,x(k));
end
hold on
plot(x, EvalPA, '-x', 'LineWidth',0.7, 'MarkerSize',7)
plot(x, EvalPS, '-x', 'LineWidth',0.7, 'MarkerSize',7)
legend('Padé approximant', 'Truncated power series', 'FontName','CMU
Serif', 'FontWeight','bold')
xlabel('x', 'FontName','CMU Serif', 'FontWeight','bold')
```

Question 5

Program used to produce Figure 5.1

```
L = [5 10 50 100 150];
Lmax = 150;
c = ones(2*Lmax + 1,1);
c(2) = 0.5;
c(3) = -0.125;
for k=3:2*Lmax
    prod = 1/8;
    for i=0:k-3
        prod = prod*(3 + 2*i)/(2*(i+3));
    end
    c(k+1) = ((-1)^(k-1))*(prod);
end
for j = 1:numel(L)
    [p, q, R] = ProgramA(c,L(j),L(j),0,x);
    zeros = ProgramB(flip(p));
    poles = ProgramB(flip([1;q]));
    zeros = real(zeros(imag(zeros) == 0));
    poles = real(poles(imag(poles) == 0));
    zeros = zeros(real(zeros)<-1);
    poles = poles(real(poles)<-1);
    subplot(numel(L), 2, 2*j -1)
    hold on
    plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
    plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
    title(append('L = ',num2str(L(j))), 'FontName','CMU Serif',
'FontWeight','bold')
    ax = gca;
    ax.XAxisLocation = 'origin';
    ax.YAxisLocation = 'origin';
    ax.XLim = [-10 0];
    xlabel('Re(x)', 'FontName', 'CMU Serif', 'FontWeight','bold',
'FontSize', 8);
    ylabel('Im(x)', 'FontName', 'CMU Serif', 'FontWeight','bold','FontSize',
8);

    subplot(numel(L), 2, 2*j)
    hold on
    plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
    plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
    title(append('L = ',num2str(L(j))), 'FontName','CMU Serif',
'FontWeight','bold')
    ax = gca;
    ax.XAxisLocation = 'origin';
    ax.YAxisLocation = 'origin';
    ax.XLim = [-1.5 -0.9];
    xlabel('Re(x)', 'FontName', 'CMU Serif', 'FontWeight','bold',
'FontSize', 8);
    ylabel('Im(x)', 'FontName', 'CMU Serif', 'FontWeight','bold','FontSize',
8);
end
legend('zeros', 'poles', 'FontName', 'CMU Serif', 'FontWeight','bold')
```

Program used to produce Figure 5.2

```
L = [10 25 50 100];
Lmax = 150;
c = ones(2*Lmax + 1,1);
c(2) = 0.5;
c(3) = -0.125;
for k=3:2*Lmax
    prod = 1/8;
    for i=0:k-3
        prod = prod*(3 + 2*i)/(2*(i+3));
    end
    c(k+1) = ((-1)^(k-1))*(prod);
end
for j = 1:numel(L)
    [p, q, R] = ProgramA(c,L(j),L(j),0,x);
    zeros = ProgramB(flip(p));
    poles = ProgramB(flip([1;q]));
    subplot(2, 2, j)
    hold on
    plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
    plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
    title(append('L = ',num2str(L(j))), 'FontName','CMU Serif',
'FontWeight','bold')
    ax = gca;
    ax.XAxisLocation = 'origin';
    ax.YAxisLocation = 'origin';
    ax.XLim = [-2 2];
    xlabel('Re(x)', 'FontName','CMU Serif', 'FontWeight','bold',
'FontSize', 8);
    ylabel('Im(x)', 'FontName','CMU Serif', 'FontWeight','bold','FontSize',
8);
end
legend('zeros', 'poles', 'FontName','CMU Serif', 'FontWeight','bold')
```

Program used to produce Figure 5.3

```
Lmax = 150;
L = [5 10 50 100 150];
c = ones(2*Lmax + 1,1);
c(2) = -0.5;
for k=2:2*Lmax
    c(k+1) = (-1)*(2*k - 1)/(2*k)*c(k);
end
for j = 1:numel(L)
    [p, q, R] = ProgramA(c,L(j),L(j),0,x);
    zeros = ProgramB(flip(p));
    poles = ProgramB(flip([1;q]));
    zeros = real(zeros(imag(zeros) == 0));
    poles = real(poles(imag(poles) == 0));
    zeros = zeros(real(zeros)<-1);
    poles = poles(real(poles)<-1);
    subplot(numel(L), 2, 2*j -1)
    hold on
    plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
    plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
    title(append('L = ',num2str(L(j))), 'FontName','CMU Serif',
'FontWeight','bold')
    ax = gca;
    ax.XAxisLocation = 'origin';
```

```

    ax.YAxisLocation = 'origin';
    ax.XLim = [-10 0];
    xlabel('Re(z)', 'FontName', 'CMU Serif', 'FontWeight', 'bold',
'FontSize', 8);
    ylabel('Im(z)', 'FontName', 'CMU Serif', 'FontWeight', 'bold', 'FontSize',
8);

    subplot(numel(L), 2, 2*j)
    hold on
    plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
    plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
    title(append('L = ', num2str(L(j))), 'FontName', 'CMU Serif',
'FontWeight', 'bold')
    ax = gca;
    ax.XAxisLocation = 'origin';
    ax.YAxisLocation = 'origin';
    ax.XLim = [-1.5 -0.9];
    xlabel('Re(x)', 'FontName', 'CMU Serif', 'FontWeight', 'bold',
'FontSize', 8);
    ylabel('Im(x)', 'FontName', 'CMU Serif', 'FontWeight', 'bold', 'FontSize',
8);
end
legend('zeros', 'poles', 'FontName', 'CMU Serif', 'FontWeight', 'bold')

```

Program used to produce Figure 5.4

```

L = [5 10 50 100];
Lmax = 150;
c = ones(2*Lmax + 1, 1);
for k=1:2*Lmax
    c(k+1) = 1/factorial(k);
end

for j = 1:numel(L)
    [p, q, R] = ProgramA(c, L(j), L(j), 0, x);
    zeros = ProgramB(flip(p));
    poles = ProgramB(flip([1; q]));
    subplot(2, 2, j)
    hold on
    plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
    plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
    title(append('L = ', num2str(L(j))), 'FontName', 'CMU Serif',
'FontWeight', 'bold')
    ax = gca;
    ax.XAxisLocation = 'origin';
    ax.YAxisLocation = 'origin';
    xlabel('Re(x)', 'FontName', 'CMU Serif', 'FontWeight', 'bold',
'FontSize', 8);
    ylabel('Im(x)', 'FontName', 'CMU Serif', 'FontWeight', 'bold', 'FontSize',
8);
end
legend('zeros', 'poles', 'FontName', 'CMU Serif', 'FontWeight', 'bold')

```

Program used to produce Figure 5.5

```
Lmax = 30;
L = [5 10 20 30];
syms x
f(x) = exp(x)/(1+x);
c = ones(2*Lmax + 1,1);
c(1) = f(0);
for k=2:2*Lmax+1
    Df = diff(f,k-1);
    c(k) = Df(0)/factorial(k-1);
end

for j = 1:numel(L)
    [p, q, R] = ProgramA(c,L(j),L(j),0,x);
    zeros = ProgramB(flip(p));
    poles = ProgramB(flip([1;q]));
    subplot(2,2,j)
    hold on
    plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
    plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
    title(append('L = ',num2str(L(j))), 'FontName','CMU Serif',
'FontWeight','bold')
    ax = gca;
    ax.XAxisLocation = 'origin';
    ax.YAxisLocation = 'origin';
    xlabel('Re(x)', 'FontName', 'CMU Serif', 'FontWeight','bold',
'FontSize', 8);
    ylabel('Im(x)', 'FontName', 'CMU Serif', 'FontWeight','bold','FontSize',
8);
end
legend('zeros', 'poles', 'FontName','CMU Serif', 'FontWeight','bold')
```

Program used to produce Figure 5.6

```
Lmax = 40;
syms x
f(x) = exp(x)/(1+x);
c = ones(2*Lmax + 1,1);
c(1) = f(0);
for k=2:2*Lmax+1
    Df = diff(f,k-1);
    c(k) = Df(0)/factorial(k-1);
end
[p, q, R] = ProgramA(c,Lmax,Lmax,0,x);
zeros = ProgramB(flip(p));
poles = ProgramB(flip([1;q]));
hold on
plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
title(append('L = ',num2str(Lmax)), 'FontName','CMU Serif',
'FontWeight','bold')
ax = gca;
ax.XAxisLocation = 'origin';
ax.YAxisLocation = 'origin';
xlabel('Re(x)', 'FontName', 'CMU Serif', 'FontWeight','bold', 'FontSize',
8);
ylabel('Im(x)', 'FontName', 'CMU Serif', 'FontWeight','bold','FontSize', 8);
ax.XLim = [-2 2];

legend('zeros', 'poles', 'FontName','CMU Serif', 'FontWeight','bold')
```


Program used to produce Figure 5.7

```
L = [5 15 30 40];
Lmax = 40;
syms x
f(x) = (1 + x + x^2)^(1/2);
c = ones(2*Lmax + 1,1);
c(1) = f(0);
for k=2:2*Lmax+1
    Df = diff(f,k-1);
    c(k) = Df(0)/factorial(k-1);
end
for j = 1:numel(L)
    [p, q, R] = ProgramA(c,L(j),L(j),0,x);
    zeros = ProgramB(flip(p));
    poles = ProgramB(flip([1;q]));
    subplot(2,2,j)
    hold on
    plot(real(zeros), imag(zeros), 'x', 'MarkerSize', 9, 'LineWidth', 1)
    plot(real(poles), imag(poles), '.', 'MarkerSize', 9, 'LineWidth', 2)
    title(append('L = ',num2str(L(j))), 'FontName','CMU Serif',
'FontWeight','bold')
    ax = gca;
    ax.XAxisLocation = 'origin';
    ax.YAxisLocation = 'origin';
    ax.XLim = [-3 1];
    xlabel('Re(x)', 'FontName','CMU Serif', 'FontWeight','bold',
'FontSize', 8);
    ylabel('Im(x)', 'FontName','CMU Serif', 'FontWeight','bold','FontSize',
8);
end
legend('zeros', 'poles','FontName','CMU Serif', 'FontWeight','bold')
```

Program used to produce Figure 5.8

```
Lmax = 30;

syms x
f(x) = (1 + x + x^2)^(1/2);
c = ones(2*Lmax + 1,1);
c(1) = f(0);
for k=2:2*Lmax+1
    Df = diff(f,k-1);
    c(k) = Df(0)/factorial(k-1);
end
x = -8:0.01:4;

L = 10;
[~, ~, R1] = ProgramA(c,L,L,0,x);
subplot(2,2,1)
hold on
plot(x, sqrt(1+x+x.^2), 'Color', "#77AC30")
plot(x, -sqrt(1+x+x.^2), 'color', "#A2142F")
plot(-8:0.01:-2.28, R1(1:573), 'color', 'b')
plot(-2.27:0.01:4, R1(574:1201), 'color', 'b')
ax = gca;
ax.XAxisLocation = 'origin';
ax.YAxisLocation = 'origin';
xlabel('x','FontName','CMU Serif', 'FontWeight','bold', 'FontSize', 8);
ylim([-15 15])
```

```

title(append('L = ',num2str(L)), 'FontName','CMU Serif',
'FontWeight','bold', 'FontSize', 12)

L = 20;
[~, ~, R2] = ProgramA(c,L,L,0,x);
subplot(2,2,2)
hold on
plot(x, sqrt(1+x+x.^2), 'Color', "#77AC30")
plot(x, -sqrt(1+x+x.^2), 'color', "#A2142F")
plot(x, R2, 'color', 'b')
ax = gca;
ax.XAxisLocation = 'origin';
ax.YAxisLocation = 'origin';
xlabel('x','FontName','CMU Serif', 'FontWeight','bold', 'FontSize', 8);
ylim([-15 15])
title(append('L = ',num2str(L)), 'FontName','CMU Serif',
'FontWeight','bold', 'FontSize', 12)

L = 25;
[~, ~, R3] = ProgramA(c,L,L,0,x);
subplot(2,2,3)
hold on
plot(x, sqrt(1+x+x.^2), 'Color', "#77AC30")
plot(x, -sqrt(1+x+x.^2), 'color', "#A2142F")
plot(-8:0.01:-2.56, R3(1:545), 'color', 'b')
plot(-2.55:0.01:4, R3(546:1201), 'color', 'b')
ax = gca;
ax.XAxisLocation = 'origin';
ax.YAxisLocation = 'origin';
xlabel('x','FontName','CMU Serif', 'FontWeight','bold', 'FontSize', 8);
ylim([-15 15])
title(append('L = ',num2str(L)), 'FontName','CMU Serif',
'FontWeight','bold', 'FontSize', 12)

L = 30;
[~, ~, R4] = ProgramA(c,L,L,0,x);
subplot(2,2,4)
hold on
plot(x, sqrt(1+x+x.^2), 'Color', "#77AC30")
plot(x, -sqrt(1+x+x.^2), 'color', "#A2142F")
plot(x, R4, 'color', 'b')
ax = gca;
ax.XAxisLocation = 'origin';
ax.YAxisLocation = 'origin';
xlabel('x','FontName','CMU Serif', 'FontWeight','bold', 'FontSize', 8);
ylim([-15 15])
title(append('L = ',num2str(L)), 'FontName','CMU Serif',
'FontWeight','bold', 'FontSize', 12)

legend('+|f_6(x)|', '-|f_6(x)|', 'Pade Approximant', 'FontName','CMU
Serif', 'FontWeight','bold', 'FontSize', 12)

```

